

INF264 – Project 2

Malware Classification for Santa

Deadline: Sunday October 18th, 23:59
Deliver your submission at MittUiB

General Information

Projects are a compulsory part of this course. **The project can be done either alone or in pairs.** If you decide to work in pairs, add a paragraph to your report explaining the division of labor. Note that both students will get the same grade, regardless of the division of labor. Grading will mainly be based on the following three qualities:

- **Correctness.** Your answers are correct and your code works as expected.
- **Clarity of code.** Your code is easy to read and understand for someone who has not seen it before. Use meaningful variable names and comments where necessary.
- **Reporting.** Your report is well-structured and clearly written.

Especially, weight is put on model selection, evaluation procedures, and the quality of the report when grading.

Deliverables

You should deliver **exactly two files**¹:

1. A detailed **PDF report** containing an explanation of your approach, design choices and results. The main point of the report is to show that you have understood the task and that you can explain your approach and results in a clear and concise manner. See section B.2 for more details about the report.
2. A **ZIP file** containing your code. We may want to run your code if we think it is necessary to confirm that it works the way it should. Please include a `README.txt` file in your ZIP file that briefly explains how we should run your code. Note that all the numbers that you report should be reproducible from the code that you submit. See section B.1 for more details about the code.

Programming languages. Please submit your code in Python. Notebooks, standalone scripts, or a combination of both are all acceptable. If you want to use another programming language, please ask us first.

¹The reason we ask you **not** to put the report inside the ZIP file is that in this way, the graders can use MittUiB's Speedgrader-functionality. So please make graders' work easier and follow the instructions.

Code of Conduct

All code and the report delivered should be your own work. In other words, you are not allowed to directly copy code from online tutorials, public code repositories etc. If you take inspiration from other sources, you should clearly state this in your report (and add a comment in the code). Regarding generative AI and language models in particular, see the section *"On the use of LLMs"* on the course page at MittUiB.

You are allowed to use machine learning libraries.² You are, of course, also free to use libraries such as `matplotlib`, `numpy` and `pandas` for tasks such as data analysis, data manipulation and visualisation. If you are unsure whether something is allowed or not, please ask us first.

Late Submission Policy

All late submissions will get a deduction of 2 points. In addition, there is a 2-point deduction for every starting 12-hour period. That is, a project submitted at 00:01 on October 19th will get a 4-point deduction and a project submitted at 12:01 on the same day will get a 6-point deduction (and so on). All projects submitted on October 21st or later are automatically failed. Executive summary: Submit your project on time. The late penalties are designed to be harsh enough so that nobody should benefit by returning their work late. There will be no possibility to resubmit failed projects.

1 Tasks

Learning goals: Learn a proper process for selecting and evaluating classifiers and learn to experiment with ensembles in conjunction with supervised methods.



Figure 1: Santa's workshop. (Image by Freepik.com)

Scenario: As December draws near, Santa has been busy preparing for the gift delivery deadline. However, something has gone terribly wrong in the systems responsible for correct gift deliveries. The Chief Elf Officer (CEO) Blimy of Santa's Workshop has reached out to *you*, a prominent machine learning expert, for assistance.

²Using `sklearn` should be sufficient to complete this project. If you are ambitious about deep learning, you can consider to use libraries such as, for example, `PyTorch`.

In the last month ÅpenKI has been busy training their new and most advanced yapper yet: snakkGPT. However, while training has been going on ÅpenKI has found that it has accidentally hacked into Santa's IT infrastructure and infected many of Santa's key gift delivery systems. In an interview ÅpenKI's head Mas Namtla said "There really should have been some laws against that.". Blimy is particularly worried as he doesn't know exactly which systems have been infected and which are fine. This could lead to significant delays in the gift distribution if all systems have to be manually checked.

To address the issue, Santa and Blimy have decided to invest in an automated malware detection setup that not only finds out which systems are infected, but also classifies the type of malware implemented. Blimy has prepared a dataset containing images of all the code running on Santa's systems, which need to be classified. This is where you come in to help.

There are 10 different types of classes that need to be detected: 1 class for safe code, and 9 different classes for infected code. See section A for more details about the dataset(s) you will be working with.

Problem 1: Automatic Malware Classifier

Your main goal is to **design and build a reliable classifier** that can take input images from the code, and correctly identify whether the code is safe or belongs to a specific class of malware. You are allowed to use existing implementations such as, for example, `sklearn`.

You should try (at least) **three different types of classifiers**³ and perform model selection including hyperparameter tuning. You should also compute an estimate of the expected performance on new data for your final classifier. How you choose to measure the performance of your classifier is up to you, but you should justify your choices.

Problem 2: Ensemble Methods

Sometimes two (or more) heads can work better than one. Create an ensemble of classifier using the best models of each type found in the model selection process in Problem 1 and use majority voting to predict the classes. Here you can either use a method from `sklearn`, such as `sklearn.ensemble.VotingClassifier`, or implement your own ensemble voting. Evaluate the performance of the ensemble against the final classifier from the previous task. Analyze the results that you are seeing and reason about why your ensemble is either giving you improved, or worse, results when compared to the best classifier found in Problem 1.

Of course, Santa also expects you to provide a **detailed report** for both problems explaining your work, including results and the reasons behind your design choices (see section B.2 for more details about the report). You don't want to end up on Santa's naughty list!

³For our purposes, two classifiers are of different type if they were covered in different lectures.

A Dataset Description

Download the dataset⁴ `dataset.npz` from MittUiB. You can load the dataset using NumPy as follows:

```
dataset = np.load("dataset.npz")
X, y = dataset["X"], dataset["y"]
```

The NumPy array `X` has shape $(n, 1024)$ consisting of n grayscale images of size 32×32 pixels, and `y` has shape $(n,)$. A single image `X[k]` is a NumPy array with shape $(1024,)$ taking values in the range $0 - 255$ where 0 is black and 255 is white. Note that the images are flattened into a 1-dimensional array. If you need to convert a flattened image `X[k]` into a 2-dimensional array, you can call `X[k].reshape(32,32)`. See fig. 2 for an illustration of reshaping a NumPy array.

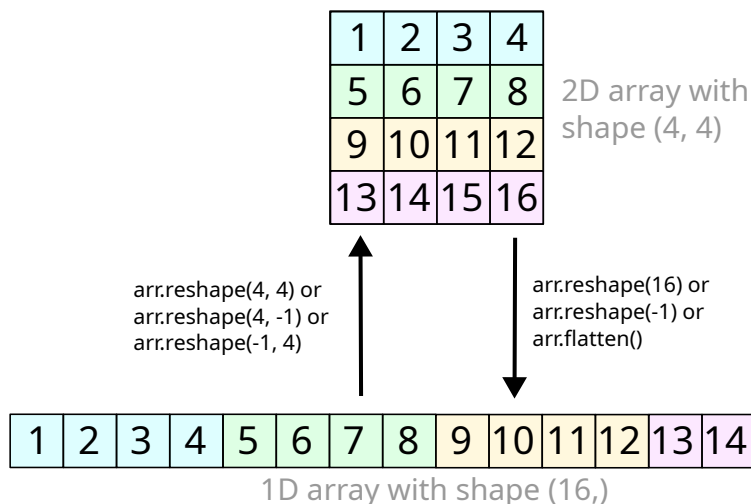


Figure 2: Illustration showing how NumPy array’s `reshape()` method works. Note that `flatten()` returns a copy of the array, whereas `reshape()` tries to return a new view if possible.

To visualise the image `X[k]` you can use the following code:

```
import matplotlib.pyplot as plt
plt.imshow(X[k].reshape(32,32), vmin=0, vmax=255, cmap="gray")
plt.show()
```

The labels `y` consist of integers in the range $0 - 9$ encoding the class each image belongs to. The different labels are listed in table 1.

⁴The dataset for this project is adapted from the *virus-mnist* dataset, available at <https://www.kaggle.com/datasets/datamunge/virusmnist>.

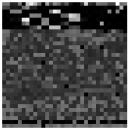
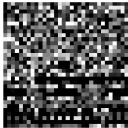
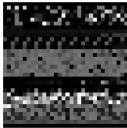
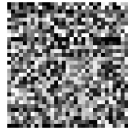
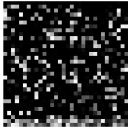
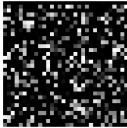
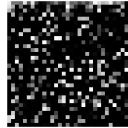
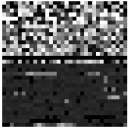
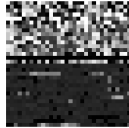
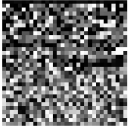
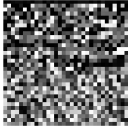
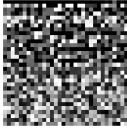
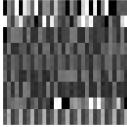
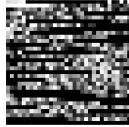
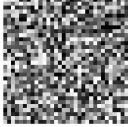
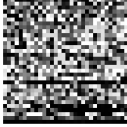
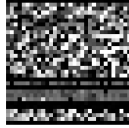
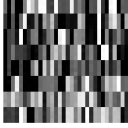
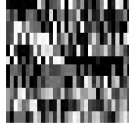
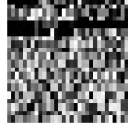
Class Name	Label	Example images					
Good	0						
Adware	1						
Trojan 1	2						
Trojan 2	3						
Installer	4						
Backdoor 1	5						
Crypto	6						
Backdoor 2	7						
Downloader	8						
Heuristic	9						

Table 1: The first column lists the different classes we want to differentiate between. The middle column lists the corresponding labels as they appear in y . The last column shows ten examples images for each class.

B Guidelines and Suggestions

B.1 Code and Implementation

The following list contain some points you should consider when implementing your solution:

- ☐ **Data exploration:** What can you say about the dataset, and how does it affect your design choices? What is the class distribution? Do not be afraid of actually looking at some of the images in the training dataset, either.
- ☐ **Metrics:** Decide on how you want to measure the goodness of a classifier. This should be based on the task at hand, and the dataset.
- ☐ **Pre-processing:** Do you need to do any pre-processing of the data?
- ☐ **Model selection:** You should try **at least 3 different types** of classifiers. Experiment with hyperparameters. It is crucial to have correct model selection and evaluation procedures. Based on your model selection procedure, automatically select the best model. Do not touch the test set until you have selected your final model.
- ☐ **Quality assessment:** Remember to analyse your results. Perform sanity checks. How well does your selected model perform on unseen data? What are the weaknesses of your model? Consider confusion matrices and computing precision/recall for the different classes to gain insight⁵.
- ☐ **Visualisation:** Can you produce other insightful plots or other types of visualisations? Can you manually inspect some of the images that your classifier misclassifies?
- ☐ **Reproducibility:** Your results should be reproducible, i.e., one should be able to easily run your code and get the same numbers that you give in your report. Thus, you should write an automated test pipeline that runs all of your tests (given enough time). That is, training and assessment of all models and hyperparameters. If you use Jupyter notebooks, make sure that your results can be reproduced after restarting the kernel and running all cells in order.

Constraints. Model selection and evaluation should not take unreasonable amounts of time. If it takes more than 30 minutes to run your code, you should consider reducing the complexity of your model, use a subset of the data for model selection (but train the final model on the full dataset) or tune fewer hyperparameters. All code should be CPU-based (no GPU acceleration).

B.2 Report

The report should consist of two parts, a **summary** and a **technical report**, both in the same PDF.

Summary

The summary should give a short, non-technical overview of your project. You should also argue, based on your results, whether the machine learning approach is appropriate for this task and what is your expectation of its performance in real-life.

⁵See, for example, `confusion_matrix()` and `classification_report()` from `sklearn.metrics`.

Technical report

The technical report should cover what you have actually done and why. It should contain detailed information on your design choices and experimental design. The list below is a guideline for what you should include in your report (but you are free to structure it as you see fit):

- ☐ Your observations about the dataset.
- ☐ Pre-processing steps (if any).
- ☐ Choice of candidate models and hyperparameters. And why were the others omitted?
- ☐ Chosen performance measure. Justify your choice.
- ☐ Model selection scheme(s) that you used. Justify your choices
- ☐ What is your final classifier, and how does it work? Justify why you think it is the best choice.
- ☐ How well it is expected to perform in production (on new data). Justify your estimate.
- ☐ How did you implement the ensemble, and how does it work? Justify your choice of classifiers.
- ☐ Measures taken to avoid overfitting.
- ☐ Given more resources (time or compute), how would you improve your solution?
- ☐ Present your results using tables and figures (and not screenshots of console output).

Remember, **our main goal is learning**, so it is perfectly fine to report failed experiments. Especially if you can explain why things did not work as you initially expected. Figures and plots are expected. Also note that whenever you report performance measures, **clearly state which data set you used** to compute them (training, validation, test).